

Dinámica y movimiento para videojuegos y gamificación

Tema 5. Colisiones

Pedro Ángel Bernaola Galván

Febrero 2026

Contenido

- 1 Introducción
- 2 Conservación de la energía
- 3 Conservación del momento lineal y angular
- 4 Control de colisiones

1 Introducción

Las Colisiones

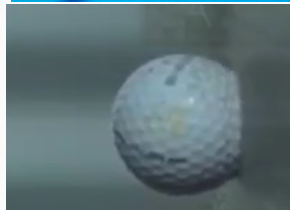
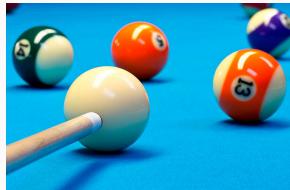
En el desarrollo de videojuegos, modelar el impacto entre objetos es fundamental para el realismo. Una colisión se define por:

- El intercambio de momento lineal y angular.
- La duración del impacto (habitualmente muy breve).
- La conservación o pérdida de energía cinética.
- El **Momento Lineal** ($\vec{p}$), y el **Momento Angular** ($\vec{L}$) se definen como:

$$\vec{p} = m \cdot \vec{v} \quad \vec{L} = I \cdot \vec{\omega}$$

donde m es la masa y $\vec{v}$ la velocidad del objeto e I y $\vec{\omega}$ el momento de inercia y la velocidad angular del objeto.

- $\vec{p}$ y $\vec{L}$ se conservan si no hay fuerzas exteriores
- **La energía no siempre se conserva**



`bloque_muelle.py`

La Complejidad de las Colisiones Reales

Implementar una colisión robusta desde cero es una tarea compleja que va más allá de aplicar leyes de conservación.

- **Detección de contacto:** Determinar el instante exacto y el punto de colisión (*collision detection*) requiere algoritmos geométricos costosos.
- **Gestión de la energía y el momento:** Además de conservar $\vec{p}$, hay que tratar la pérdida de energía (e) y la transferencia al momento angular.
- **Motores de física:** Afortunadamente, herramientas como Pymunk o PyBullet ofrecen una simulación de contacto muy superior a otros modelos (como el rozamiento del aire), evitando retoques manuales constantes.

2 Conservación de la energía

Coeficiente de Restitución (e)

Define cómo de **elástica** es una colisión. Se calcula mediante la relación de velocidades relativas antes y después del impacto:

$$e = -\frac{v_{2f} - v_{1f}}{v_{2i} - v_{1i}}$$

Valores típicos de e :

- $e = 1$: **Colisión perfectamente elástica** (se conserva la energía).
- $e = 0$: **Colisión perfectamente inelástica** (los objetos se pegan).
- $0 < e < 1$: **Colisiones reales** (fricción, deformación, calor).



Video. Cuna de Newton

`bolas01.py`

Elasticidad en Pymunk (Coeficiente de Restitución)

En Pymunk, la restitución se controla con la propiedad `elasticity` de los objetos `Shape`.

- Representa la capacidad de un objeto para conservar energía cinética tras un choque.
- Los valores se sitúan normalmente en el rango $[0, 1]$.
- **0.0 (Inelástico):** El objeto no rebota (como plastilina).
- **1.0 (Perfectamente elástico):** El objeto conserva toda su velocidad normal.
- El **coeficiente de restitución del choque** se define como el producto de las propiedades `elasticity` de los dos objetos.
- Se admite `elasticity > 1`: por ejemplo cuando se salta sobre una cama elástica
- Si la colisión es muy rápida, es conveniente hacer *substepping*

`bolas01.py`

`bloque_muelle.py`

Elasticidad en PyBullet

PyBullet gestiona la colisión elástica mediante el parámetro **restitution** en el método **changeDynamics**.

- **Asignación:** Se aplica a cada *link* o cuerpo de forma independiente.
- **Interacción:** A diferencia de otros motores, PyBullet requiere que **ambos** cuerpos definan una restitución > 0 para que el rebote sea visible.
- **Estabilidad:** Por defecto, el motor puede anular rebotes de baja energía para mantener la estabilidad del sistema (*contact damping*).
- **Paso de tiempo:** Para colisiones de alta velocidad y alta elasticidad, hay problemas igual que en Pymunk, se recomienda aumentar la frecuencia de muestreo: `p.setTimeStep(1/240.0)`

Problemas con el solapamiento de objetos

Debido a que el tiempo es discreto un cuerpo puede superponerse con otro antes de que se detecte la colisión. En el motor Pymunk, el mecanismo para resolver este problema se denomina **Bias**.

- **collision_bias**: Define qué porcentaje del error de solapamiento se corrige en cada paso. Si es muy alto, los objetos se separan de forma explosiva.
- **collision_slop**: Pymunk introduce además un “margen de tolerancia” (slop). Los objetos pueden penetrarse hasta esta distancia antes de que el *bias* empiece a actuar. Si es bajo, más precisión pero tiene mayor coste de cálculo y puede provocar vibraciones (correcciones continuas)

Implementación de `collision_slop` y `collision_bias`

Para gestionar la penetración, estos parámetros se ajustan en objeto `space`:

- `space.collision_slop` (El margen):
 - Si la penetración es menor al *slop*, el motor la ignora.
 - Ayuda a evitar que los objetos vibren (*jitter*) al estar en reposo.
- `space.collision_bias` (La corrección): Determina cómo de rápido se resuelve el solapamiento que supera al *slop*.
 - Es un porcentaje: 0,1 significa corregir el 10% del exceso en cada paso.

Ejemplo:

```
1 space = pymunk.Space()
2 space.collision_slop = 0.1 # si pones 0.05, mas precision, menos margen
3 space.collision_bias = 0.8 # si pones 0.05, separacion mas rapida
4
```

ERP en PyBullet: El homólogo del *Bias*

PyBullet utiliza el **ERP** (*Error Reduction Parameter*), que conceptualmente cumple la misma función que el `collision_bias` de Pymunk.

- **Fracción de corrección:** Al igual que en Pymunk, el ERP determina qué parte de la penetración se resuelve por cada *step*.
- **Rango de valores:**
 - **0.0:** No se corrige la posición (los objetos se hunden).
 - **1.0:** Intento de corrección total en un solo *step* (causa inestabilidad y explosiones físicas).
 - **0.2 a 0.8:** Valores estándar para un equilibrio entre rigidez y estabilidad.
- **Estabilidad:** Se mantienen las mismas reglas: valores cercanos a 1.0 producen el mismo efecto de “explosión” o inestabilidad que vimos en Pymunk.

¿Dónde se cambia el ERP en PyBullet?

Diferencia de implementación: Mientras que en Pymunk suele ser un ajuste más global, en PyBullet el ERP es muy flexible y puede configurarse:

- A nivel global del motor físico para los contactos:
`p.setPhysicsEngineParameter(contactERP=0.8)`
- Individualmente para cada restricción (*cid*) o unión:
`p.changeConstraint(cid, erp=0.5)`
- **Contact ERP:** Un valor alto simula materiales muy rígidos (acero); un valor bajo simula contactos más “blandos” o permisivos.

3 Conservación del momento lineal y angular

Conservación del Momento Lineal

A partir de la Segunda Ley de Newton, $\vec{F} = \frac{d\vec{p}}{dt}$, si la fuerza externa neta sobre un sistema es nula ($\vec{F}_{ext} = 0$), entonces:

$$\frac{d\vec{p}_{tot}}{dt} = 0 \implies \vec{p}_{tot} = \text{constante}$$

Esta ley es universal: se cumple en cualquier colisión, independientemente de si es elástica o inelástica, ya que las fuerzas de interacción interna (Tercera Ley de Newton) siempre se cancelan por pares:

- “El momento lineal total de un **sistema aislado** permanece constante en el tiempo”.
- Esto implica que el centro de masas del sistema mantiene su estado de movimiento uniforme, ya que $\boxed{\vec{p}_{tot} = M\vec{v}_{cm}}$.

Ejemplo: Si me empujas, te empujo

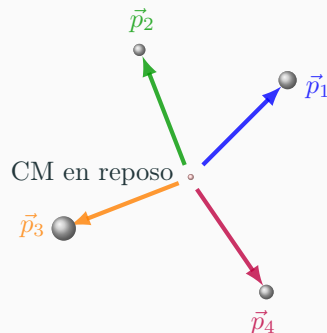
- Un coche acelerando *empuja* el suelo hacia atrás y el suelo lo empuja hacia delante (`coche_plataforma.py`)
- Cuando un objeto choca contra una pared ¿cuándo empuja más si el choque es elástico o inelástico? (`bola_pared.py`)
- ¿Qué pasa al chocar contra un objeto inamovible? (`bola_pared_infinita.py`)

¡¡CUIDADO!! Si es Pymunk el que gestiona la interacción, ya se encarga de que se cumpla la **Tercera Ley de Newton** y de que cada fuerza de *acción* tenga su correspondiente *reacción*. Si aplicamos fuerzas externas manualmente, tenemos que asegurarnos de que cumplimos esta condición.

Ejemplo: La Física de una Explosión

Consideremos una bomba inicialmente en reposo ($\vec{v}_{cm} = 0$).
Tras la explosión, las fuerzas internas proyectan fragmentos en todas direcciones:

- El momento total inicial es cero, por lo que la suma vectorial de los momentos de todos los fragmentos debe seguir siendo cero:
- $$\sum m_i \vec{v}_i = 0$$
- Los fragmentos se reparten de forma que el centro de masas del conjunto sigue en reposo en el mismo punto exacto de la explosión.
 - Si el conjunto se movía con velocidad rectilínea y constante, el CM también seguirá con velocidad rectilínea constante.



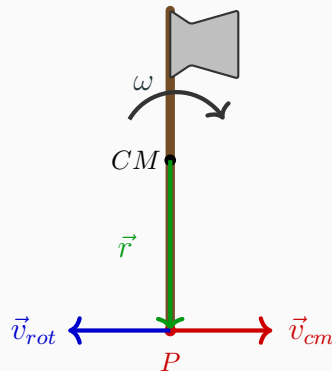
cm.py
cm_parbola.py
baston.py

Lanzamiento del hacha: El punto de reposo

Para que un punto P del hacha actúe como eje de giro (CIR), su velocidad en el sistema del laboratorio debe ser nula o igual a la velocidad de avance deseada.

$$\vec{v}_P = \vec{v}_{cm} + \vec{v}_{rot} \implies \vec{v}_{cm} = \vec{v}_P - (\vec{\omega} \times \vec{r}_{cm \rightarrow P})$$

- Si el mango está orientado hacia abajo ($r_y > 0$) y el giro es horario ($\omega > 0$).
- La rotación empuja el extremo hacia la **izquierda**.
- Por tanto, debemos **restar** ese vector (o sumar su opuesto) para que el CM absorba ese movimiento y el punto P parezca pivotar.

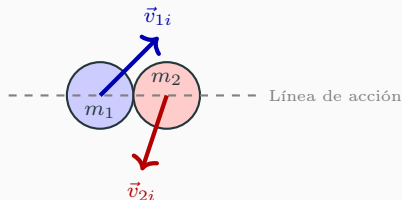


$$\text{Si } |\vec{v}_{cm}| = |\vec{v}_{rot}|, \vec{v}_P = 0$$

Colisiones Oblicuas (2D)

En la mayoría de los juegos, los objetos no chocan perfectamente centrados.

1. Identificar la **línea de acción** (el vector normal al contacto).
2. Descomponer las velocidades iniciales en componentes **normales** y **tangenciales**.
3. La colisión sólo afecta a las **componentes normales**.
4. Las componentes tangenciales permanecen constantes (si no hay fricción).



`colision_oblicua02.py`

Colisiones con Fricción y Rotación

Cuando un objeto choca con una superficie o con otro objeto, la fricción en el punto de contacto genera un **torque**. Este torque cambia el momento angular ($\vec{L}$) del objeto, haciendo que comience a girar o cambie su rotación previa.

- El cambio en el momento lineal tangencial depende del coeficiente de fricción estática y cinética.
- Si el impacto no es central, la fuerza genera una rotación instantánea.
- Además hay transferencia de momento lineal en una dirección distinta de la línea de acción

Ejemplo: si no hay fricción y una bola está inicialmente parada, después de la colisión se moverá en la dirección de la línea de acción. Si hay fricción esto ya no ocurre así.

`colision_oblicua02.py`

Momento Angular de un Sistema y su Conservación

Sistemas de Partículas Para un conjunto de masas, el momento angular total ($\vec{L}$) es la suma vectorial de los momentos individuales:

$$\vec{L} = \sum_i \vec{r}_i \times m_i \vec{v}_i$$

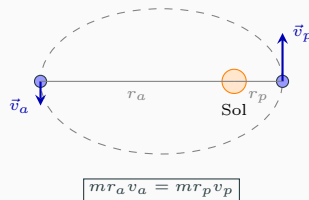
Caso del Sólido Rígido Cuando las distancias entre masas son fijas y rotan respecto a un eje, la expresión se simplifica:

$$\vec{L} = I\vec{\omega}$$

donde I es el momento de inercia del cuerpo.

Ley de Conservación $\vec{L}$ permanece constante incluso con fuerzas externas, siempre que el torque neto sea nulo ($\vec{\tau}_{ext} = \vec{0}$):

Ejemplo: Órbitas Planetarias



La fuerza gravitatoria es central (no genera torque respecto al Sol), por lo que el planeta acelera al acercarse para conservar su $\vec{L}$.

[conservacion_L.py](#)

4 Control de colisiones

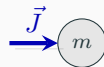
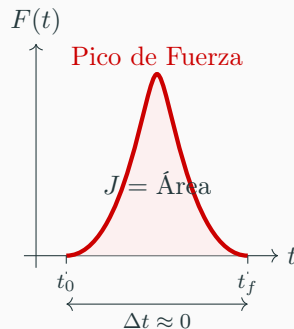
Impulso y Cambio de Momento

Definición Física El impulso ($\vec{J}$) es la medida del efecto de una fuerza $\vec{F}$ durante un intervalo Δt :

$$\vec{J} = \int_{t_0}^{t_0+\Delta t} \vec{F}(t) dt = \Delta \vec{p}$$

Naturaleza del Choque

- Son fuerzas **impulsivas**: extremas en tiempos mínimos.
- Dependen de la deformación elástica de los materiales.
- Pymunk y PyBullet **omiten la fuerza** y aplican directamente un impulso $\Delta \vec{p}$ (**básicamente un $\Delta \vec{v}$**).
- En un proceso iterativo se van añadiendo pequeños $\Delta \vec{p}$ y se comprueba que los cuerpos no se atraviesan y se aplica la ley de Coulomb de rozamiento.



Aproximación del Impulso y Fuerzas Externas

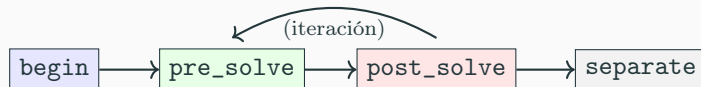
En ocasiones, no nos interesa simular la geometría del contacto, sino únicamente el efecto del intercambio de energía.

- **Aproximación del impulso:** Si el objetivo es transferir momento sin una colisión explícita entre formas (*shapes*), se recurre a modelos de fuerzas externas.
- **Implementación:** Se aplica una fuerza $\vec{F}$ que actúa durante un intervalo de tiempo determinado Δt .
- El cambio en el momento lineal se define como: $\vec{J} = \int \vec{F} dt \simeq \vec{F} \cdot \Delta t = \Delta \vec{p}$
- Esto es útil para simular explosiones cercanas, propulsores o campos de fuerza donde el contacto físico es irrelevante

Control de Colisiones: Callbacks

El motor permite intervenir la física mediante un **Collision Handler**. Existen cuatro eventos donde podemos inyectar código:

- **begin**: Se llama una única vez cuando dos formas se tocan por primera vez.
Retorno: **True** para procesar el choque, **False** para ignorarlo.
- **pre_solve**: Se llama en cada *step* mientras dura el contacto. Permite cambiar parámetros como la fricción o elasticidad justo antes del cálculo.
- **post_solve**: Se llama tras el cálculo físico. Es el momento ideal para leer la fuerza aplicada o destruir objetos (ej. una bala que desaparece).
- **separate**: Se llama cuando los objetos dejan de tocarse.



Puentear el Comportamiento por Defecto

Para "puentear" la física y aplicar impulsos manuales, la estrategia es:

1. Clasificamos los tipos de objetos que van a aparecer en la simulación asignándoles un `self.shape.collision_type` (ej. 1 para la bola blanca, 2 para las bolas de color y 3 para el taco)
2. Definimos un handler para los tipos de colisión:
`space.on.collision(tipo1,tipo2, begin=mi_funcion_begin)` Si es necesario se hace también para `pre_solve`, `post_solve` y/o `separate`
3. Implementar la lógica en `begin` o `pre_solve`.
4. **IMPORTANTE:** Retornar `False`.

Al retornar `False`, el *Solver* de Pymunk descarta el cálculo de fuerzas normal y tangencial, permitiéndonos aplicar nuestro propio impulso sin interferencias.

Ejemplos

- En `linea_accion.py` tenemos un solo tipo de objetos que colisionan (la clase Bola). Sobreescribimos `begin` para sacar por pantalla las velocidades justo antes de la colisión y también `separate` para dejar detenida la simulación y mostrar la línea de acción.
- En `billar_taco.py`. Asignamos `self.shape.collision_type = 1` a las bolas y 2 al taco. Sobreescribimos `post_solve` para dejar el taco parado en seco.
- En `billar_taco_impulso.py` sobreescribimos `begin` para la colisión entre bola y taco y calculamos manualmente el impulso lineal y angular que añadimos a la bola.
IMPORTANTE: devuelve `False` para que no se llame al resto de funciones.